

Experiment 11 : MAP COLORING TO IMPLEMENT CSP

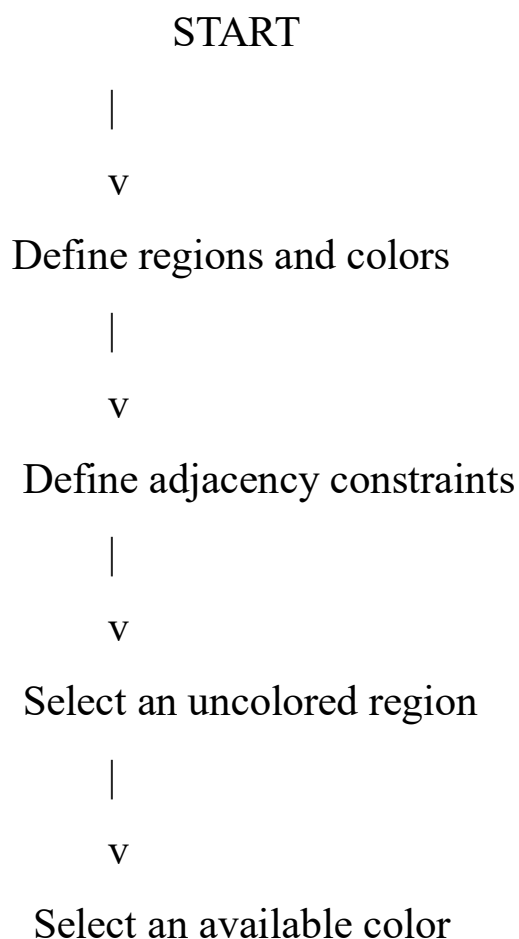
Aim

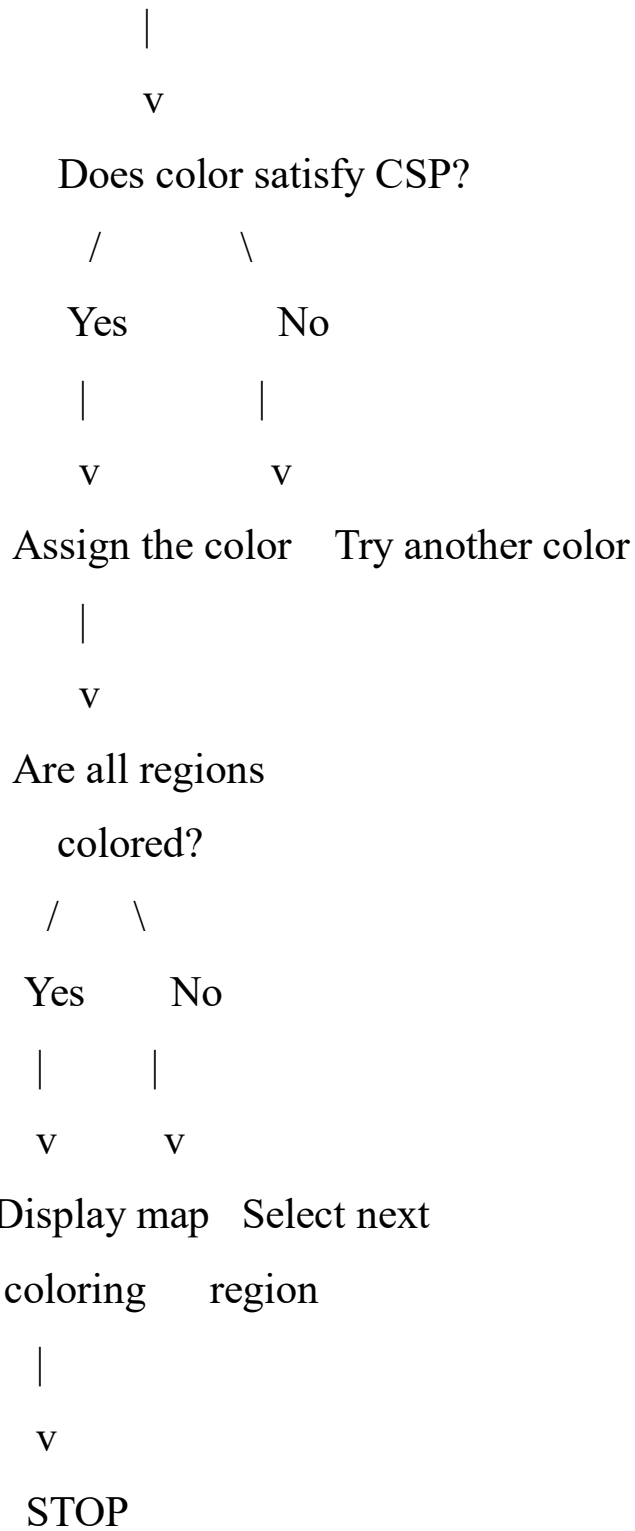
To write a Python program to solve the Map Coloring problem using **Constraint Satisfaction Problem (CSP)** techniques.

Objective

- To understand the concept of Constraint Satisfaction Problems.
- To represent map coloring as a CSP.
- To assign colors to different regions of a map.
- To ensure that no two adjacent regions have the same color.
- To implement backtracking search to find a valid coloring.

Flowchart





Code (Python)

Map Coloring using CSP and Backtracking

Define the regions

```
regions = ['A', 'B', 'C', 'D']
```

```
# Available colors
```

```
colors = ['Red', 'Green', 'Blue']
```

```
# Define neighboring regions
```

```
neighbors = {
```

```
    'A': ['B', 'C'],
```

```
    'B': ['A', 'C', 'D'],
```

```
    'C': ['A', 'B', 'D'],
```

```
    'D': ['B', 'C']
```

```
}
```

```
# Check whether assigning a color is valid
```

```
def is_valid(region, color, assignment):
```

```
    for neighbor in neighbors[region]:
```

```
        if neighbor in assignment and assignment[neighbor] == color:
```

```
            return False
```

```
    return True
```

```
# Backtracking CSP algorithm
```

```
def map_coloring(assignment):
```

```
    # If all regions are colored
```

```
if len(assignment) == len(regions):
    return assignment

# Select an uncolored region
for region in regions:
    if region not in assignment:
        break

# Try each available color
for color in colors:

    if is_valid(region, color, assignment):

        assignment[region] = color

    result = map_coloring(assignment)

    if result:
        return result

# Backtrack
del assignment[region]

return None
```

Solve the map coloring problem

```
solution = map_coloring({})
```

Display result

```
print("Map Coloring Solution:")
```

if solution:

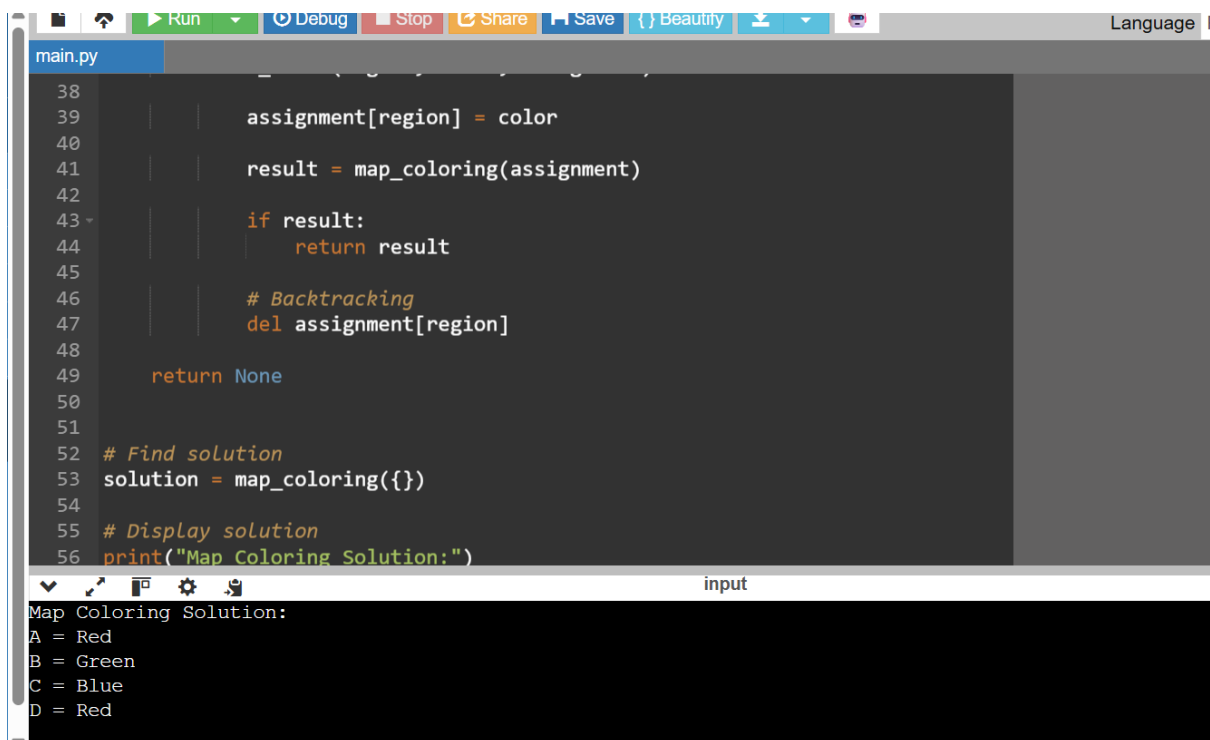
```
    for region, color in solution.items():
```

```
        print(region, "=", color)
```

else:

```
    print("No solution exists.")
```

Output



The screenshot shows a code editor with a file named `main.py`. The code implements a map coloring algorithm using backtracking. It defines a `map_coloring` function that takes an assignment of colors to regions and returns a solution if one exists. The main part of the code calls `map_coloring({})` to find a solution and prints it. The output window shows the result: "Map Coloring Solution:" followed by "A = Red", "B = Green", "C = Blue", and "D = Red".

```
38
39     assignment[region] = color
40
41     result = map_coloring(assignment)
42
43     if result:
44         return result
45
46     # Backtracking
47     del assignment[region]
48
49     return None
50
51
52 # Find solution
53 solution = map_coloring({})
54
55 # Display solution
56 print("Map Coloring Solution:")
```

Map Coloring Solution:
A = Red
B = Green
C = Blue
D = Red

Result

Thus, the Python program to solve the **Map Coloring problem using Constraint Satisfaction Problem (CSP)** was executed successfully, and a valid color assignment was obtained.

Conclusion

The experiment helped in understanding the concept of **Constraint Satisfaction Problems** and the use of **backtracking search**. The Map Coloring problem was successfully solved by assigning colors to regions while satisfying the constraint that neighboring regions must have different colors.